

GitHub & 파일 자동 정리 시스템 포트폴리오

프로그래밍입문

발표 날짜: 6/12

학번: 2026964033

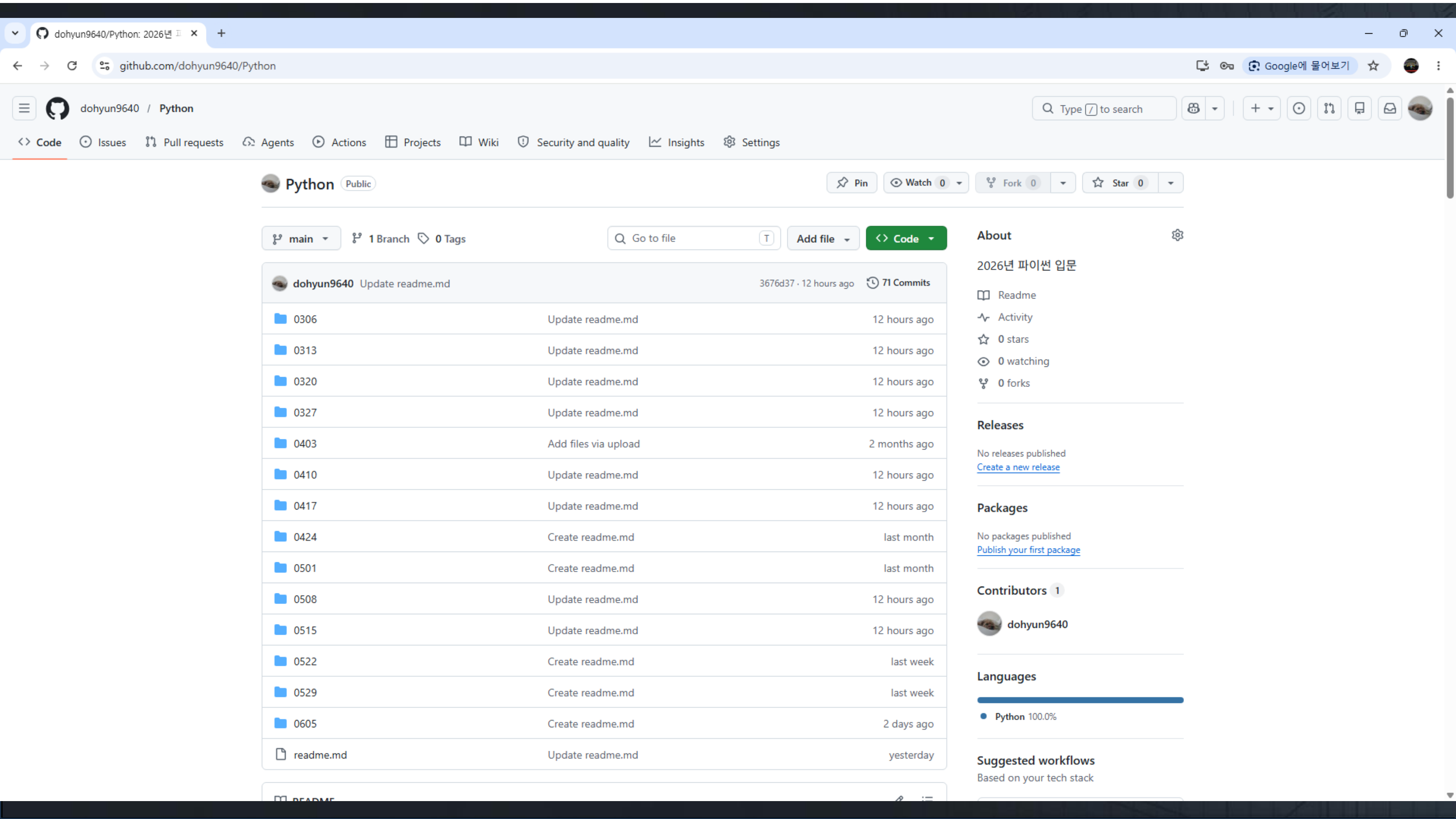
이름: 이도현

담당 교수: 김주현

깃허브 주소

<https://github.com/dohyun9640/Python>

PYTHON PROGRAMMING & VERSION CONTROL PROGRESS REPORT



Week 1: 파이썬 기초 및 개발 환경 구축

01. 이론: 파이썬 소개 및 입출력

1장 파이썬 소개 요약

컴퓨터는 하드웨어와 소프트웨어로 구성되며, 다양한 일을 할 수 있는 유연한 기계이다. 컴퓨터에게 일을 시키려면 사람이 자세한 명령어 목록을 작성해야 하는데 이것을 프로그램이라고 한다. 인공지능, 주식 자동매매, 로봇 프로세스 자동화처럼 여러 분야에서 프로그램이 활용된다. 컴퓨터는 반복적이고 지루한 작업을 빠르고 정확하게 처리하는 데 강점이 있다. 컴퓨터는 사람의 언어를 직접 이해하지 못하므로 프로그래밍 언어가 필요하다. 사람이 파이썬 같은 언어로 프로그램을 작성하면 컴파일러나 인터프리터가 이를 컴퓨터가 이해할 수 있는 기계어로 바꿔준다. 문법이 간결하고 배우기 쉬우며 결과를 바로 확인할 수 있는 인터프리터 언어라 초보자에게 적합하다. 또한 라이브러리가 풍부하고 설치가 쉬워 데이터 분석, 인공지능, 웹 개발 등 다양한 분야에 활용된다. 주피터 노트북과 Colab은 코드와 설명을 함께 작성하며 실행할 수 있는 환경이다. 기초 문법으로는 `print()` 함수, 문자열, 계산식 출력이 다뤄진다. 문자열은 큰따옴표나 작은따옴표 안에 작성해야 하며 따옴표가 없으면 오류가 발생한다. `print()` 함수는 문자열이나 계산 결과를 화면에 출력할 때 사용한다.

예를 들어:

```
print("파이썬에 오신 것을 환영합니다.")
```

```
print("2+3=", 2+3)
```

오류 처리에서는 따옴표 누락, 대소문자 오류, 괄호 오류 등이 소개된다. 파이썬은 대소문자를 구분하므로 `print`와 `Print`는 다르게 인식된다. 소스 파일의 확장자는 `.py`이다.

02. 검증: 초기 저장소 세팅 및 기초 코드 커밋

 dohyun9640 / Python

[Code](#) [Issues](#) [Pull requests](#) [Agents](#) [Actions](#) [Projects](#) [Wiki](#) [Security and quality](#) [Insights](#) [Settings](#)

Commits

History for [Python](#) / 0306 on [main](#) All users All time

Commits on Mar 12, 2026

Add files via upload

 dohyun9640 authored on Mar 12

Verified 09e6a59   

Commits on Mar 11, 2026

Add files via upload

 dohyun9640 authored on Mar 11

Verified 8054935   

Commits on Mar 6, 2026

Create readme.md

 dohyun9640 authored on Mar 6

Verified 8474fc0   

End of commit history for this file

Name	Last commit message	Last commit date
..		
bug.py	Add files via upload	3 months ago
debugged.py	Add files via upload	3 months ago
hello.py	Add files via upload	3 months ago
lab1.py	Add files via upload	3 months ago
lab2.py	Add files via upload	3 months ago
programming 1-1.py	Add files via upload	3 months ago
programming 1-2.py	Add files via upload	3 months ago
programming 1-3.py	Add files via upload	3 months ago
programming 1-4.py	Add files via upload	3 months ago
readme.md	Update readme.md	35 minutes ago
도전문제1-1.py	Add files via upload	3 months ago
도전문제1-2.py	Add files via upload	3 months ago

Week 2: 데이터의 할당과 연산

01. 이론: 변수, 수식 및 자료형 변환

2장 변수와 수식 요약

변수는 값을 저장하기 위해 이름을 붙인 메모리 공간이다. 파이썬에서는 변수에 값을 대입하면 자동으로 변수가 만들어진다.

```
x = 100
y = 200
sum = x + y
print(sum)
```

파이썬의 변수는 값을 직접 담는 상자라기보다 메모리에 저장된 객체를 가리키는 이름에 가깝다. 변수값은 다시 대입하면 바뀔 수 있다.

산술 연산자로는 +, -, *, /, //, %, ** 등이 있다. 예를 들어 **는 거듭제곱, %는 나머지, //는 몫을 구한다.

또한 연산자에는 우선순위가 있다. 곱셈과 나눗셈이 덧셈과 뺄셈보다 먼저 계산되며, 괄호를 사용하면 우선순위를 바꿀 수 있다.

사용자 입력은 input() 함수로 받는다. 입력값은 기본적으로 문자열이므로 숫자 계산을 하려면 int()나 float()로 형 변환해야 한다.

```
x = int(input("숫자를 입력하세요: "))
```

이 장에서는 BMI 계산, 거듭제곱 계산, 거스름돈 계산 같은 예제를 통해 변수, 입력, 출력, 연산식을 연습했다.

02. 검증: 연산자 활용 및 사용자 입력 실습 적용

Week 2: 데이터의 할당과 연산

Files

main

Go to file

0306

0313

0320

0327

0403

0410

0417

0424

0501

0508

0515

0522

0529

0605

readme.md

Name	Last commit message	Last commit date
..		
2장 문제 정답.txt	Add files via upload	3 months ago
area.py	Add files via upload	3 months ago
asmd.py	Add files via upload	3 months ago
atm.py	Add files via upload	3 months ago
bmi.py	Add files via upload	3 months ago
char_bot.py	Add files via upload	3 months ago
inputFunc.py	Add files via upload	3 months ago
invest.py	Add files via upload	3 months ago
light_speed.py	Add files via upload	3 months ago
programming 2-1.py	Add files via upload	3 months ago
programming 2-10.py	Add files via upload	3 months ago
programming 2-2.py	Add files via upload	3 months ago
programming 2-3.py	Add files via upload	3 months ago
programming 2-4.py	Add files via upload	3 months ago
programming 2-5.py	Add files via upload	3 months ago
programming 2-6.py	Add files via upload	3 months ago
programming 2-7.py	Add files via upload	3 months ago
programming 2-8.py	Add files via upload	3 months ago
programming 2-9.py	Add files via upload	3 months ago
readme.md	Update readme.md	39 minutes ago

Python / 0313 /

↑ Top

readme.md	Update readme.md	39 minutes ago
robot.py	Add files via upload	3 months ago
shpere.py	Add files via upload	3 months ago
sum.py	Add files via upload	3 months ago
tempcony.py	Add files via upload	3 months ago
triangleArea.py	Add files via upload	3 months ago
volume.py	Add files via upload	3 months ago
도전문제 2-1.py	Add files via upload	3 months ago
도전문제 2-2.py	Add files via upload	3 months ago
도전문제 2-3.py	Add files via upload	3 months ago
도전문제 2-4.py	Add files via upload	3 months ago
도전문제 3-1.py	Add files via upload	3 months ago
중간점검 2-1-1.py	Add files via upload	3 months ago
중간점검 2-1-2.py	Add files via upload	3 months ago
중간점검 2-2-1.py	Add files via upload	3 months ago
중간점검 2-2-2.py	Add files via upload	3 months ago
중간점검 2-2-3.py	Add files via upload	3 months ago
중간점검 2-3-1.py	Add files via upload	3 months ago
중간점검 2-3-2.py	Add files via upload	3 months ago

Week 3: 제어 흐름의 설계 (조건문)

01. 이론: if-elif-else 및 논리 연산자

3장 조건문 요약

조건문은 프로그램이 상황에 따라 다르게 동작하도록 만드는 제어문이다. 조건문이 없다면 프로그램은 항상 같은 순서로만 실행된다.

기본 구조는 if, else, elif이다.

```
if score >= 60:
    print("합격")
else:
    print("불합격")
```


조건식에는 관계 연산자가 사용된다. 대표적으로 $>$, $<$, $>=$, $<=$, $=$, $!=$ 가 있다.

여러 조건을 함께 판단할 때는 논리 연산자를 사용한다.

```
and # 둘 다 참이면 참
or  # 하나라도 참이면 참
not # 참과 거짓을 반대로 바꿈
```



파이썬에서는 들여쓰기로 코드 블록을 구분한다. 따라서 조건문 안에서 실행될 문장은 반드시 같은 깊이로 들여쓰기해야 한다.

조건문은 단순 if문, if-else문, 중첩 if문, 연속 if-elif-else문으로 확장할 수 있다. 예제에서는 할인 가격 계산, 지진 규모에 따른 피해 설명, 숫자 비교 등을 다뤘다.

02. 검증: 분기 처리 로직 구현 및 다중 파일 형상 관리

 dohyun9640 / Python

Q Type  to search

 ▼

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Commits

History for Python / 0320 on main

All usersAll time

Commits on Mar 25, 2026

Add files via upload

 dohyun9640 authored on Mar 25

Verifiedea25d52

Commits on Mar 13, 2026

Create readme.md

 dohyun9640 authored on Mar 13

Verified67ce296

End of commit history for this file

Week 3: 제어 흐름의 설계 (조건문)

Python / 0320 / Add file ▾ ⋮			Python / 0320 / ↑ Top		
dohyun9640 Update readme.md			3f5911a · 40 minutes ago 🕒 History		
Name			Last commit message		
Last commit date					
..					
3장 텍스트 정답.txt			Add files via upload		
4장 텍스트 정답.txt			Add files via upload		
coin_tossing.py			Add files via upload		
earthquake.py			Add files via upload		
factorial.py			Add files via upload		
magic_ball.py			Add files via upload		
mathprob.py			Add files via upload		
programming 1.py			Add files via upload		
programming 10.py			Add files via upload		
programming 11.py			Add files via upload		
programming 12.py			Add files via upload		
programming 13.py			Add files via upload		
programming 14.py			Add files via upload		
programming 15.py			Add files via upload		
programming 16.py			Add files via upload		
programming 2.py			Add files via upload		
programming 3.py			Add files via upload		
programming 4.py			Add files via upload		
programming 5.py			Add files via upload		
programming 6.py			Add files via upload		
programming 7.py			Add files via upload		
programming 8.py			Add files via upload		
programming 9.py			Add files via upload		
readme.md			Update readme.md		
shipping 137p.py			Add files via upload		
shipping.py			Add files via upload		
soccer.py			Add files via upload		
user_input.py			Add files via upload		
water_state.py			Add files via upload		
도전문제 3-1 125P.py			Add files via upload		
도전문제 3-2-1 127p.py			Add files via upload		
도전문제 3-2-2 127p.py			Add files via upload		
도전문제 3-3 133p.py			Add files via upload		
도전문제 3-4 134p.py			Add files via upload		
도전문제 3-5 142p.py			Add files via upload		
도전문제 3-6 143p.py			Add files via upload		
도전문제 3-7 144p.py			Add files via upload		
도전문제 4-1 169p.py			Add files via upload		
중간점검 3-4 126p.py			Add files via upload		
중간점검 4-2-1 162p.py			Add files via upload		
중간점검 4-2-2 162p.py			Add files via upload		
중간점검 4-2-3 162p.py			Add files via upload		

Week 4: 프로세스의 자동화 (반복문)

01. 이론: for, while 루프 및 range 구조

4장 반복문 요약

반복문은 같은 작업을 여러 번 실행할 때 사용한다. 반복적이고 단순한 작업은 컴퓨터가 매우 잘 처리하므로 자동화에 적합하다.

반복문에는 크게 두 가지가 있다.

for문은 반복 횟수를 알고 있을 때 사용한다.

```
for i in range(5):
    print("방문을 환영합니다!")
```

range() 함수는 일정한 범위의 숫자를 만들어준다.

```
range(5)      # 0부터 4까지
range(1, 6)   # 1부터 5까지
range(1, 10, 2) # 1, 3, 5, 7, 9
```

while문은 조건이 참인 동안 반복한다.

```
while count < 5:
    print(count)
    count += 1
```

반복문에서는누적합, 카운트, 숫자 맞추기 게임 같은 프로그램을 만들 수 있다. 조건이 계속 참이면 무한 루프가 발생할 수 있으므로 종료 조건을 잘 설정해야 한다. 또한 반복문 안에 반복문을 넣는 중첩 반복문도 배운다. 구구단, 별 출력, 표 형태 출력 등에 활용했다.

02. 검증: 반복 작업을 통한 무한 루프 제어 및 누적합 구현

 dohyun9640 / Python



Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Commits

History for `Python` / 0327 on `main`

All users

All time

Commits on Apr 2, 2026

Add files via upload

 dohyun9640 authored on Apr 2

Verified

47d64ba



Commits on Mar 20, 2026

Create readme.md

 dohyun9640 authored on Mar 20

Verified

befe5d8



End of commit history for this file

Week 4: 프로세스의 자동화 (반복문)

	dohyun9640	Update readme.md	cc68ca3 · 41 minutes ago	History
Name	Last commit message	Last commit date		
..				
4장 텍스트 문제.txt	Add files via upload	2 months ago		
Programming1.py	Add files via upload	2 months ago		
Programming10.py	Add files via upload	2 months ago		
Programming11.py	Add files via upload	2 months ago		
Programming12.py	Add files via upload	2 months ago		
Programming13.py	Add files via upload	2 months ago		
Programming14.py	Add files via upload	2 months ago		
Programming15.py	Add files via upload	2 months ago		
Programming16.py	Add files via upload	2 months ago		
Programming2.py	Add files via upload	2 months ago		
Programming3.py	Add files via upload	2 months ago		
Programming4.py	Add files via upload	2 months ago		
Programming5.py	Add files via upload	2 months ago		
Programming6.py	Add files via upload	2 months ago		
Programming7.py	Add files via upload	2 months ago		
Programming8.py	Add files via upload	2 months ago		
Programming9.py	Add files via upload	2 months ago		

도전문제 2-1 177p.py	Add files via upload	2 months ago
도전문제 3-1 179p.py	Add files via upload	2 months ago
도전문제 4-1 180p.py	Add files via upload	2 months ago
도전문제 4-2 180p.py	Add files via upload	2 months ago
도전문제 5-1 181p.py	Add files via upload	2 months ago
도전문제 6-1 184p.py	Add files via upload	2 months ago
도전문제 7-1 185p.py	Add files via upload	2 months ago
도전문제 8-1 188p.py	Add files via upload	2 months ago
도전문제 9-1 190p.py	Add files via upload	2 months ago
도전문제 9-2 190p.py	Add files via upload	2 months ago
중간점검 2-1 183.py	Add files via upload	2 months ago
중간점검 3-2 189p.py	Add files via upload	2 months ago

break.py	Add files via upload	2 months ago
cal_pi.py	Add files via upload	2 months ago
countinue.py	Add files via upload	2 months ago
dice.py	Add files via upload	2 months ago
else.py	Add files via upload	2 months ago
gambler.py	Add files via upload	2 months ago
guess.py	Add files via upload	2 months ago
gugu.py	Add files via upload	2 months ago
inf_loop.py	Add files via upload	2 months ago
math.py	Add files via upload	2 months ago
money.py	Add files via upload	2 months ago
nested1.py	Add files via upload	2 months ago
nested2.py	Add files via upload	2 months ago
nested3.py	Add files via upload	2 months ago
password.py	Add files via upload	2 months ago
prime.py	Add files via upload	2 months ago
readme.md	Update readme.md	42 minutes ago
res.py	Add files via upload	2 months ago
root.py	Add files via upload	2 months ago
sum.py	Add files via upload	2 months ago
도전문제 10-1 194p.py	Add files via upload	2 months ago

Week 5: 코드의 모듈화 및 재사용 (함수)

01. 이론: def 정의, 매개변수 및 반환값(return)

5장 함수 요약

함수는 특정 작업을 수행하는 명령어들의 모음에 이름을 붙인 것이다. 반복해서 사용하는 코드를 함수로 만들면 프로그램을 더 깔끔하고 효율적으로 작성할 수 있다.

함수는 def로 정의한다.

```
def hello():  
    print("안녕하세요")
```

함수를 실행하려면 함수 이름을 호출한다.

```
hello()
```

함수에는 값을 전달할 수 있는데, 이를 매개변수라고 한다.

```
def greet(name):  
    print(name, "님 안녕하세요")
```

함수는 결과값을 돌려줄 수도 있다. 이때 return을 사용한다.

```
def add(x, y):  
    return x + y
```

함수는 코드 재사용, 프로그램 구조화, 오류 감소에 도움을 준다. 이 장에서는 사각형 그리기, 막대그래프 그리기, 계산 함수 만들기 등을 통해 함수의 필요성과 사용법을 익혔다.

02. 검증: 커스텀 함수 작성 및 코드 재사용성 확보

dohyun9640 / Python

code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

🔍 Type / to search

Commits

History for Python / 0410 on main

🔍 All users

📅 All time

🔍 Commits on Apr 17, 2026

Add files via upload

Verified

9759a60

dohyun9640 authored on Apr 17

🔍 Commits on Apr 10, 2026

Create readme.md

Verified

cfebd1

dohyun9640 authored on Apr 10

🔍 End of commit history for this file

Week 5: 코드의 모듈화 및 재사용 (함수)

Python / 0410 /

Add file ▾

...

dohyun9640

Update readme.md

043a8e7 · 42 minutes ago

History

Name	Last commit message	Last commit date
..		
5장 텍스트 문제.md	Add files via upload	2 months ago
bisection.py	Add files via upload	2 months ago
cal_pay.py	Add files via upload	2 months ago
conv_temp.py	Add files via upload	2 months ago
pizza.py	Add files via upload	2 months ago
programming 1.py	Add files via upload	2 months ago
programming 10.py	Add files via upload	2 months ago
programming 11.py	Add files via upload	2 months ago
programming 12.py	Add files via upload	2 months ago
programming 13.py	Add files via upload	2 months ago
programming 14.py	Add files via upload	2 months ago
programming 2.py	Add files via upload	2 months ago
programming 3.py	Add files via upload	2 months ago
programming 4.py	Add files via upload	2 months ago
programming 5.py	Add files via upload	2 months ago
programming 6.py	Add files via upload	2 months ago
programming 7.py	Add files via upload	2 months ago
programming 8.py	Add files via upload	2 months ago
programming 9.py	Add files via upload	2 months ago
readme.md	Update readme.md	42 minutes ago
return.py	Add files via upload	2 months ago
welcome.py	Add files via upload	2 months ago

Week 6: 연속형 데이터 구조의 이해 (리스트)

01. 이론: 리스트 인덱싱, 슬라이싱 및 내장 메서드

6 파이썬 자료구조 | : 리스트 요약

리스트는 여러 개의 값을 순서대로 저장하는 자료구조이다. 대괄호 []를 사용해 만든다.

```
scores = [10, 20, 60, 70, 80]
```

리스트의 각 값은 요소라고 하며, 위치 번호인 인덱스를 이용해 접근한다. 인덱스는 0부터 시작한다.

```
scores[0] # 첫 번째 요소
```

리스트는 값을 변경할 수 있다.

```
scores[0] = 100
```

음수 인덱스를 사용하면 뒤에서부터 접근할 수 있다.

```
scores[-1] # 마지막 요소
```

리스트에는 다양한 연산이 있다.

```
append() # 추가
insert() # 원하는 위치에 삽입
remove() # 특정 값 삭제
pop() # 마지막 또는 특정 위치 삭제
sort() # 정렬
len() # 길이
max() # 최댓값
min() # 최솟값
sum() # 합계
```

리스트의 일부를 잘라내는 것을 슬라이싱이라고 한다.

```
scores[1:4]
```

또한 리스트 안에 리스트를 넣으면 2차원 리스트를 만들 수 있다. 표, 행렬, 좌석 배치 같은 자료를 표현할 때 사용했다. 이 장에서는 성적을 입력받아 평균, 최댓값, 최솟값, 특정 점수 이상인 학생 수를 구하는 프로그램을 만들었다.

02. 검증: 1차원 및 2차원 리스트를 활용한 데이터 집합 처리

 dohyun9640 / Python

[Code](#) [Issues](#) [Pull requests](#) [Agents](#) [Actions](#) [Projects](#) [Wiki](#) [Security and quality](#) [Insights](#) [Settings](#)

Commits

History for [Python](#) / 0417 on [main](#)

 All users  All time

 Commits on Apr 22, 2026

Add files via upload

 dohyun9640 authored on Apr 22

Verified

2c35220

 Commits on Apr 19, 2026

Add files via upload

 dohyun9640 authored on Apr 19

Verified

ee87172

 Commits on Apr 17, 2026

Create readme.md

 dohyun9640 authored on Apr 17

Verified

0f225f6

 End of commit history for this file

Week 6: 연속형 데이터 구조의 이해 (리스트)

Python / 0417 /

Add file ▾

⋮

dohyun9640

Update readme.md

37fd1fd · 43 minutes ago

🕒

History

Name	Last commit message	Last commit date
📁 ..		
📄 6장 텍스트 문제.md	Add files via upload	2 months ago
📄 checker.py	Add files via upload	2 months ago
📄 friends.py	Add files via upload	2 months ago
📄 gym_score1.py	Add files via upload	2 months ago
📄 gym_score2.py	Add files via upload	2 months ago
📄 list_add.py	Add files via upload	2 months ago
📄 list_comp.py	Add files via upload	2 months ago
📄 list_comp2.py	Add files via upload	2 months ago
📄 programming 1.py	Add files via upload	2 months ago
📄 programming 10.py	Add files via upload	2 months ago
📄 programming 11.py	Add files via upload	2 months ago
📄 programming 12.py	Add files via upload	2 months ago
📄 programming 13.py	Add files via upload	2 months ago
📄 programming 2.py	Add files via upload	2 months ago
📄 programming 3.py	Add files via upload	2 months ago
📄 programming 4.py	Add files via upload	2 months ago
📄 programming 5.py	Add files via upload	2 months ago

Python / 0417 /

↑ Top

📄 programming 6.py	Add files via upload	2 months ago
📄 programming 7.py	Add files via upload	2 months ago
📄 programming 8.py	Add files via upload	2 months ago
📄 programming 9.py	Add files via upload	2 months ago
📄 pytha_tri.py	Add files via upload	2 months ago
📄 readme.md	Update readme.md	43 minutes ago
📄 salary.py	Add files via upload	2 months ago
📄 second1.py	Add files via upload	2 months ago
📄 second2.py	Add files via upload	2 months ago
📄 stack.py	Add files via upload	2 months ago
📄 tic_tac_toe.py	Add files via upload	2 months ago
📄 tranpose1.py	Add files via upload	2 months ago
📄 tranpose2.py	Add files via upload	2 months ago

Week 7: 복합 자료구조의 활용 (튜플, 세트, 딕셔너리)

01. 이론: 각 자료구조의 특성 및 문자열 시퀀스 연산

7장 자료구조 || 요약

이 장에서는 리스트 외의 자료구조인 튜플, 세트, 딕셔너리, 문자열 연산을 배운다. 튜플은 리스트와 비슷하지만 한 번 만들면 값을 변경할 수 없다.

```
fruits = ("apple", "banana", "grape")
```

튜플은 변경되면 안 되는 데이터를 저장할 때 사용한다. 항목이 하나인 튜플을 만들 때는 쉼표가 필요하다.

```
single = ("apple",)
```

세트는 중복을 허용하지 않고 순서가 없는 자료구조이다.

```
s = {1, 2, 3}
```

세트는 중복 제거, 집합 연산에 유용하다. 합집합, 교집합, 차집합 등을 구할 수 있다. 딕셔너리는 키와 값의 쌍으로 데이터를 저장한다.

```
phone = {"KIM": "123-4567"}
```

키를 이용해 값을 찾을 수 있으므로 연락처, 사전, 회원 정보처럼 이름과 값이 연결된 자료를 저장할 때 좋다.

```
phone["KIM"]
```

문자열도 시퀀스 자료형이므로 인덱싱, 슬라이싱, 반복, 검색이 가능하다. 문자열 메서드로는 upper(), lower(), split(), replace(), find() 등이 있다. 이 장에서는 연락처 추가, 삭제, 검색, 출력 기능을 가진 간단한 연락처 관리 프로그램을 만들었다.

02. 검증: 연락처 관리 프로그램 등 복합 데이터 모델링

dohyun9640 / Python

Search Type to search

CodeIssuesPull requestsAgentsActionsProjectsWikiSecurity and qualityInsightsSettings

Commits

History for Python / 0508 on main

Commits on May 13, 2026

Add files via upload

dohyun9640 authored 3 weeks ago

Verifiedc312b30

Commits on May 5, 2026

Create readme.md

dohyun9640 authored on May 5

Verified0ab26f5

End of commit history for this file

Week 7: 복합 자료구조의 활용 (튜플, 세트, 딕셔너리)

Python / 0508 /

Add file ▾ ⋮

dohyun9640

Update readme.md

0651c84 · 44 minutes ago ⌚ History

Name	Last commit message	Last commit date
..		
7장 텍스트 문제.md	Add files via upload	3 weeks ago
acronym.py	Add files via upload	3 weeks ago
address.py	Add files via upload	3 weeks ago
ch_count.py	Add files via upload	3 weeks ago
counter.py	Add files via upload	3 weeks ago
email.py	Add files via upload	3 weeks ago
eng_dic.py	Add files via upload	3 weeks ago
otp.py	Add files via upload	3 weeks ago
palin.py	Add files via upload	3 weeks ago
programming1.py	Add files via upload	3 weeks ago
programming10.py	Add files via upload	3 weeks ago
programming11.py	Add files via upload	3 weeks ago
programming12.py	Add files via upload	3 weeks ago
programming13.py	Add files via upload	3 weeks ago
programming14.py	Add files via upload	3 weeks ago
programming15.py	Add files via upload	3 weeks ago
programming16.py	Add files via upload	3 weeks ago

Python / 0508 /

↑ Top

programming2.py	Add files via upload	3 weeks ago
programming3.py	Add files via upload	3 weeks ago
programming4.py	Add files via upload	3 weeks ago
programming5.py	Add files via upload	3 weeks ago
programming6.py	Add files via upload	3 weeks ago
programming7.py	Add files via upload	3 weeks ago
programming8.py	Add files via upload	3 weeks ago
programming9.py	Add files via upload	3 weeks ago
readme.md	Update readme.md	44 minutes ago
score.py	Add files via upload	3 weeks ago
str_ex1.py	Add files via upload	3 weeks ago
str_ex2.py	Add files via upload	3 weeks ago
twitter1.py	Add files via upload	3 weeks ago

Week 8: 객체 지향 패러다임의 도입 (클래스)

01. 이론: 클래스 설계, 인스턴스 생성, 속성 및 메서드

8장 객체와 클래스 요약

객체지향 프로그래밍은 데이터와 함수를 하나로 묶어 객체로 만들고, 객체들이 서로 상호작용하며 프로그램을 구성하는 방식이다. 객체는 속성과 동작을 가진다. 예를 들어 자동차 객체는 색상, 모델, 속도 같은 속성을 가지고, 달리기, 멈추기, 방향 바꾸기 같은 동작을 가진다. 클래스는 객체를 만들기 위한 설계도이다. 클래스로부터 만들어진 실제 객체를 인스턴스라고 한다.

```
class Car:
    def __init__(self, color, model):
        self.color = color
        self.model = model
        self.speed = 0
```

`init()`은 객체가 생성될 때 자동으로 실행되는 생성자이다. `self`는 객체 자기 자신을 의미한다. 객체 안에 들어 있는 함수는 메서드라고 한다.

```
def speed_up(self):
    self.speed += 10
```

파이썬에서는 정수, 문자열, 리스트 등 거의 모든 것이 객체이다. 예를 들어 문자열에서 `.upper()`를 호출할 수 있는 것도 문자열이 객체이기 때문이다. 객체지향의 핵심 개념으로는 캡슐화, 클래스, 객체, 속성, 메서드 등이 있다. 이 장에서는 자동차 클래스를 만들어 속도, 색상, 모델을 저장하고 속도를 변경하는 프로그램을 작성했다.

02. 검증: 캡슐화 적용 및 자동차/학생 등 현실 객체의 코드화


dohyun9640 / Python








[Code](#)
[Issues](#)
[Pull requests](#)
[Agents](#)
[Actions](#)
[Projects](#)
[Wiki](#)
[Security and quality](#)
[Insights](#)
[Settings](#)

Commits

History for [Python](#) / 0515 on [main](#)

Commits on May 19, 2026

Add files via upload

 dohyun9640 authored 3 weeks ago

Verified
b8594b7




Commits on May 13, 2026

Create readme.md

 dohyun9640 authored 3 weeks ago

Verified
4c0c7b6




End of commit history for this file

[Python](#) / [0515](#) / 

Add file 

 dohyun9640 Update readme.md

d6c2680 · 44 minutes ago
History

Name	Last commit message	Last commit date
..		
8장 텍스트 문제.md	Add files via upload	3 weeks ago
account.py	Add files via upload	3 weeks ago
circle.py	Add files via upload	3 weeks ago
dice.py	Add files via upload	3 weeks ago
dog.py	Add files via upload	3 weeks ago
programming1.py	Add files via upload	3 weeks ago
programming2.py	Add files via upload	3 weeks ago
programming3.py	Add files via upload	3 weeks ago
programming4.py	Add files via upload	3 weeks ago
programming5.py	Add files via upload	3 weeks ago
programming6.py	Add files via upload	3 weeks ago
programming7.py	Add files via upload	3 weeks ago
programming8.py	Add files via upload	3 weeks ago
readme.md	Update readme.md	44 minutes ago
student(False).py	Add files via upload	3 weeks ago
student(true).py	Add files via upload	3 weeks ago
vector.py	Add files via upload	3 weeks ago

파일자동정리 시스템

```
import os
# os 모듈은 파일과 폴더 경로를 다룰 때 사용한다
import shutil
# shutil 모듈은 파일을 이동하거나 복사할 때 사용한다
import tkinter as tk
# tkinter는 GUI 화면을 만들 때 사용하는 파이썬 기본 라이브러리이다
from tkinter import filedialog, messagebox
# filedialog는 폴더 선택 창을 띄울 때 사용하고,
# messagebox는 알림창이나 경고창을 띄울 때 사용한다
```

```
# 확장자별로 어떤 폴더에 넣을지 정해 둔 딕셔너리
# 예를 들어 .jpg, .png 파일은 "이미지" 폴더로 이동한다
```

```
file_types = {
    "이미지": [".jpg", ".png", ".gif", ".svg"],
    "문서": [".pdf", ".docx", ".txt"],
    "압축파일": [".zip", ".rar"],
    "영상": [".mp4", ".mov"]
}
```

```
# 사용자가 선택한 폴더 경로를 저장할 변수
# 처음에는 아무 폴더도 선택하지 않았기 때문에 빈 문자열로 둔다
selected_folder = ""
```

```
# 폴더 선택 버튼을 눌렀을 때 실행되는 함수
def select_folder():
    # 함수 안에서 전역 변수 selected_folder를 사용하기 위해 global을 사용한다
    global selected_folder
    # 폴더 선택 창을 띄워서 사용자가 선택한 폴더 경로를 저장한다
    selected_folder = filedialog.askdirectory()
    # 사용자가 폴더를 선택했다면 실행한다
    if selected_folder:
        # 화면에 선택된 폴더 경로를 표시한다
        folder_label.config(text=selected_folder)
```

```
# 파일 정리 시작 버튼을 눌렀을 때 실행되는 함수
def organize_files():
    # 만약 폴더를 선택하지 않았다면 경고창을 띄우고 함수를 종료한다
    if not selected_folder:
        messagebox.showwarning("경고", "먼저 정리할 폴더를 선택하세요.")
        return
    # 결과 출력창에 있던 기존 내용을 모두 지운다
    result_text.delete("1.0", tk.END)
    # 이동한 파일 개수를 세기 위한 변수
    moved_count = 0
    # 선택한 폴더 안에 있는 파일과 폴더 이름을 하나씩 가져온다
    for file_name in os.listdir(selected_folder):
        # 선택한 폴더 경로와 파일 이름을 합쳐서 전체 파일 경로를 만든다
        file_path = os.path.join(selected_folder, file_name)
        # 현재 경로가 폴더가 아니라 파일인지 확인한다
        if os.path.isfile(file_path):
            # 파일 이름에서 확장자만 분리하고 소문자로 바꾼다
            # 예: "사진.JPG" → ".jpg"
            ext = os.path.splitext(file_name)[1].lower()
            # file_types 딕셔너리에서 폴더 이름과 확장자 목록을 하나씩 가져온다
            for folder_name, extensions in file_types.items():
                # 현재 파일의 확장자가 해당 확장자 목록에 포함되어 있는지 확인한다
                if ext in extensions:
                    # 파일을 이동할 대상 폴더 경로를 만든다
                    # 예: 선택한 폴더 경로 + "이미지"
                    target_folder = os.path.join(selected_folder, folder_name)
                    # 대상 폴더가 없으면 새로 만든다
                    # exist_ok=True는 이미 폴더가 있어도 오류가 나지 않게 한다
                    os.makedirs(target_folder, exist_ok=True)
                    # 파일을 대상 폴더로 이동한다
                    shutil.move(file_path, os.path.join(target_folder, file_name))
                    # 결과 출력창에 어떤 파일이 어디로 이동했는지 표시한다
                    result_text.insert(tk.END, f"{file_name} → {folder_name}\n")
```

```
# 이동한 파일 개수를 1 증가시킨다
moved_count += 1
# 이미 맞는 분류를 찾았으므로 반복문을 종료한다
break
```

```
# 파일 정리가 끝나면 완료 알림창을 띄운다
messagebox.showinfo(
    "완료",
    f"파일 정리가 완료되었습니다.\n총 {moved_count}개의 파일을 이동했습니다."
)
```

```
# GUI 창을 생성한다
window = tk.Tk()
# GUI 창의 제목을 설정한다
window.title("파일 자동정리 시스템")
# GUI 창의 크기를 설정한다
window.geometry("500x400")

# 프로그램 제목을 표시하는 라벨을 만든다
title_label = tk.Label(
    window,
    text="파일 자동정리 시스템",
    font=("맑은 고딕", 18, "bold")
)

# 제목 라벨을 창에 배치한다
title_label.pack(pady=20)

# 프로그램 사용 방법을 알려주는 설명 라벨을 만든다
info_label = tk.Label(
    window,
    text="정리할 폴더를 선택한 뒤, 정리 시작 버튼을 누르세요.",
    font=("맑은 고딕", 10)
)

# 설명 라벨을 창에 배치한다
info_label.pack()
# 폴더 선택 버튼을 만든다
# 이 버튼을 누르면 select_folder 함수가 실행된다
select_button = tk.Button(
    window,
    text="폴더 선택",
    width=20,
    command=select_folder
)
```

```
# 폴더 선택 버튼을 창에 배치한다
select_button.pack(pady=10)

# 사용자가 선택한 폴더 경로를 보여주는 라벨을 만든다
folder_label = tk.Label(
    window,
    text="선택된 폴더 없음",
    wraplength=450
)

# 폴더 경로 라벨을 창에 배치한다
folder_label.pack()

# 파일 정리 시작 버튼을 만든다
# 이 버튼을 누르면 organize_files 함수가 실행된다
start_button = tk.Button(
    window,
    text="파일 정리 시작",
    width=20,
    command=organize_files
)

# 파일 정리 시작 버튼을 창에 배치한다
start_button.pack(pady=15)

# 파일 이동 결과를 보여줄 텍스트 출력창을 만든다
result_text = tk.Text(
    window,
    width=55,
    height=10
)

# 결과 출력창을 창에 배치한다
result_text.pack(pady=10)
# GUI 창이 바로 닫히지 않고 계속 실행되도록 한다
window.mainloop()
```

파일 자동정리 시스템

정리할 폴더를 선택한 뒤, 정리 시작 버튼을 누르세요.

폴더 선택

선택된 폴더 없음

파일 정리 시작

파일 자동정리 시스템

정리할 폴더를 선택한 뒤, 정리 시작 버튼을 누르세요.

폴더 선택

C:/Users/dohyu/OneDrive/바탕 화면/파일 자동정리

파일 정리 시작

```
1000011003.png → 이미지  
105327346_p0_master1200.jpg → 이미지  
107928480_p0_master1200.jpg → 이미지  
107948680_p0_master1200.jpg → 이미지  
109361315_p0_master1200.jpg → 이미지  
109878949_p0_master1200.jpg → 이미지  
111221480_p0_master1200.jpg → 이미지  
111734064_p0_master1200.jpg → 이미지  
111800988_p0_master1200.jpg → 이미지  
112149411_p0_master1200.jpg → 이미지
```



동영상



이미지

느낀점

이번에 파이썬 수업에서 배운 변수, 조건문, 반복문, 함수, 리스트, 딕셔너리, 클래스 같은 내용을 바탕으로 파일 정리 프로그램 GUI를 만들어 보면서, 단순히 문법을 외우는 것보다 직접 프로그램을 만들어 보는 것이 훨씬 중요하다는 것을 느꼈다.

처음에는 파일을 선택하고, 확장자별로 분류하고, 버튼을 눌렀을 때 동작하게 만드는 과정이 어렵게 느껴졌다. 하지만 조건문을 사용해 파일 종류를 구분하고, 반복문으로 여러 파일을 한 번에 처리하며, 함수를 이용해 기능별로 코드를 나누다 보니 수업에서 배운 내용이 실제 프로그램에서 어떻게 쓰이는지 이해할 수 있었다.

특히 GUI를 만들면서 사용자가 직접 버튼을 누르고 결과를 확인할 수 있게 되니, 프로그램이 더 실용적으로 느껴졌다. 단순히 콘솔에 출력하는 프로그램보다 화면을 통해 조작할 수 있는 프로그램을 만들면서 파이썬이 실제 생활 문제를 해결하는 데 사용할 수 있는 언어라는 점을 알게 되었다.

또한 파일 정리 프로그램을 만들며 프로그램은 단순히 실행만 되는 것이 아니라, 사용자가 편하게 사용할 수 있도록 구성하는 것도 중요하다는 것을 느꼈다. 버튼 이름, 입력 방식, 정리 결과 안내처럼 작은 부분도 사용자의 편리함과 관련이 있다는 점을 배웠다.

이번 프로젝트를 통해 파이썬의 기본 문법들이 서로 따로 떨어진 내용이 아니라 하나의 프로그램을 만들기 위해 함께 사용된다는 것을 알게 되었다. 앞으로는 더 다양한 기능을 추가해서 중복 파일 삭제, 날짜별 정리, 자동 정리 기능 같은 프로그램도 만들어 보고 싶다.

나의 예상 점수

GITHUB: 15/15

프로젝트 발표: 15/15

총합: 30/30

Thank you!